CSE422 Lab Task A* Search

Problem

Consider the following undirected weighted graph. The start node is S and the goal node is G.

Edge Costs

Edge	Cost
S – A	2
S – B	1
A – C	2
A – G	6
B – C	5
C – G	1

Heuristic Values

Node	S	A	B	C	G
h(n)	4	3	5	1	0

Task

(1) Complete the missing part of the Python code for A* Search given below so that it prints the correct expansion order, path and cost.

Pseudocode

```
function A_STAR(start, goal):
    OPEN   <- priority queue ordered by f(n) = g(n) + h(n)
    CLOSED <- empty set
    g[start] <- 0
    parent[start] <- NIL
    insert start into OPEN with f = h(start)

    while OPEN is not empty:
        n <- node in OPEN with the lowest f
        remove n from OPEN
        if n is in CLOSED: continue
        add n to CLOSED

        if n = goal:
            return path from start to goal (follow parent links)

        for each neighbour m of n with edge cost c(n, m):
            new_g <- g[n] + c(n, m)
            if m has no g value yet OR new_g < g[m]:
                g[m] <- new_g
                parent[m] <- n
                insert m into OPEN with f = g[m] + h(m)

    return FAILURE
```

Partial Code

Only the core part of the algorithm (the neighbour expansion step, marked TODO) is missing. Everything else is complete.

```
import heapq
# Graph as an adjacency list: node -> [(neighbour, edge_cost), ...]
graph = {
    'S': [('A', 2), ('B', 1)],
    'A': [('S', 2), ('C', 2), ('G', 6)],
    'B': [('S', 1), ('C', 5)],
    'C': [('A', 2), ('B', 5), ('G', 1)],
    'G': [('A', 6), ('C', 1)],
}

# Heuristic values h(n)
h = {'S': 4, 'A': 3, 'B': 5, 'C': 1, 'G': 0}

def reconstruct_path(parent, goal):
    path = [goal]
    while parent[path[-1]] is not None:
        path.append(parent[path[-1]])
    return path[::-1]

def a_star(start, goal):
    open_list = [(h[start], start)]   # (f, node)
    g = {start: 0}                    # best known cost from start
    parent = {start: None}
    closed = set()
    order = []                        # expansion order

    while open_list:
        f, node = heapq.heappop(open_list)
        if node in closed:
            continue
        closed.add(node)
        order.append(node)

        if node == goal:
            return reconstruct_path(parent, goal), g[goal], order
        # ==================== TODO ====================
        # Loop over every neighbour of 'node'.
        # Compute the new g value through 'node'.
        # If it is better than the known g value (or the
        # neighbour has not been seen yet), update g and
        # parent, then push (f, neighbour) into open_list.
        # ==============================================
        pass

    return None, float('inf'), order


path, cost, order = a_star('S', 'G')
print("Expansion order:", " -> ".join(order))
print("Path:", " -> ".join(path))
print("Cost:", cost)
```

Expected Output

```
Expansion order: S -> ... -> G
Path: S -> ... -> G
Cost: ?
```